

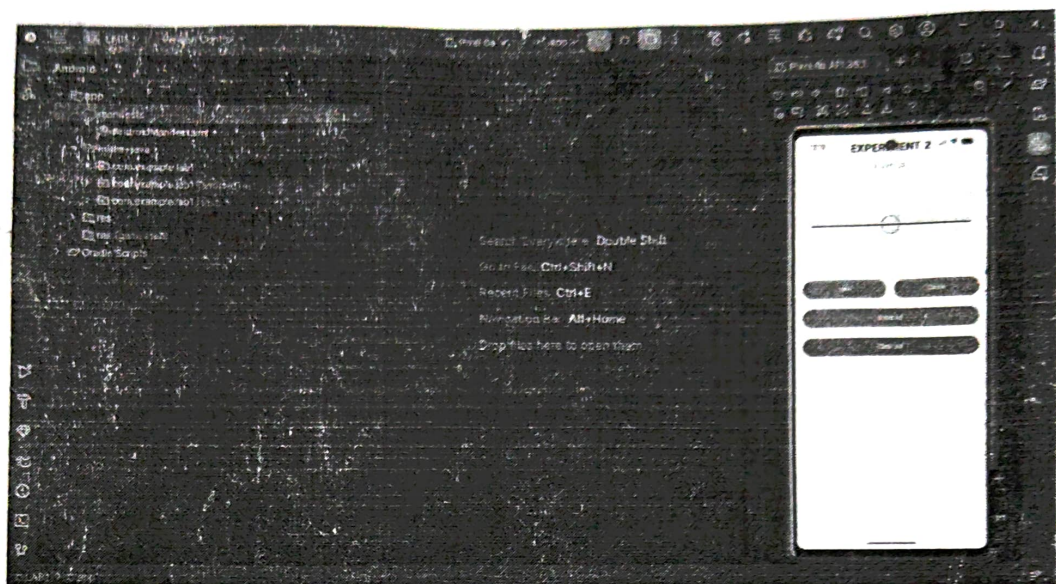
Experiment Name: Drawing shapes on canvas using custom view in Android

Aim

To create an Android application demonstrating custom drawing using canvas with interactive shape controls to understand how to render 2D graphics programmatically

Algorithm

1. Open Android Studio and create a new Android project with an empty activity template.
2. Name the project & set the package name as com.example.lob.
3. Create a new Kotlin class that extends view and override the `onDraw()` method.
4. Inside `onDraw()`, create a paint object and set properties like color, stroke width and style.
5. Use the canvas object passed to `onDraw()` to draw shapes such as lines & circles.
6. In the main layout XML file, add the custom view along with four buttons labeled Line, Circle, Show All, & Clear All.
7. In MainActivity, get references to all buttons and the custom view using `findViewById`.
8. Maintain boolean flags to track which shapes should be visible.
9. Set `onClick` listener for each button to update the flags accordingly.
10. Call `invalidate()` on the custom view after each button click to trigger a redraw.



11. In the `onDraw()` method, click the flags and only the shapes that are currently active

12. Run the application on an emulator or physical device & verify each button works correctly

Output

The application displays a white canvas area at the top of the screen. A horizontal blue line and red circle are drawn on the canvas.

Below the canvas, four buttons are displayed. Pressing Line shows only the line. Pressing circle shows only the circle. Pressing Show All

displays both shapes simultaneously. Pressing clear Show All displays both shapes simultaneously

Pressing clear All removes all shapes from the canvas.

Result: The Android application was successfully created & executed. The custom View correctly renders geometric shapes using Canvas

Paint. The button interactions work as expected, toggling the visibility of shapes dynamically.

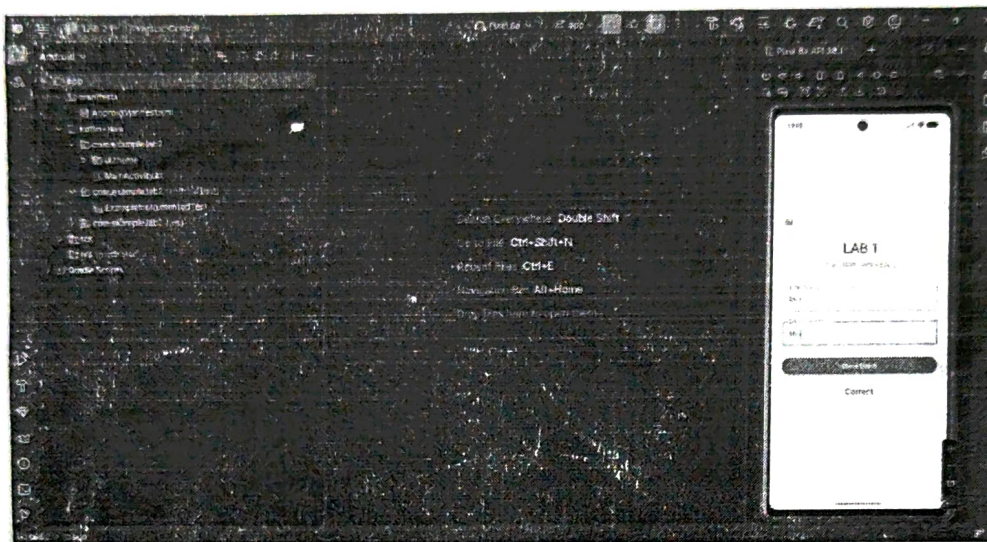
~~Exp~~ Build First Android App - Text Match Validation using Jetpack Compose

Aim:-

To build an Android application using Jetpack Compose that accepts two text inputs from the user and validates whether the entered strings match each other, demonstrating the use case of state management & composable function.

Algorithm:-

1. Open Android Studio and create a new Android project with the Empty Compose Activity template.
2. Open MainActivity.kt and set up the compose content inside the setContent block.
3. Apply the LAB2Theme to wrap the entire UI for consistent theming.
4. Create a composable function named validationApp that holds the UI logic.
5. Inside validationApp, declare two mutable state variables using remember and mutableStateOf to store the values of the text fields.
6. Declare another mutable state variable to store the result message such as correct or incorrect.
7. Use a Column composable to arrange UI elements vertically with proper padding alignment.
8. Add a Text composable at the top displaying LAB as the title and Build First Android.
9. Add two OutlinedTextField composables with labels enter first text and enter second text, binding them to the respective state variables.
10. Add two OutlinedTextField composables with labels Enter first text and Enter second text, binding them to the respective state variables.



11. Inside the button's onclick lambda, compare the 2 state variables using an equality check
12. Inside the button's onclick lambda, compare the 2 state variables using equality
13. Add a text composable below the button to display the result.
14. Build and run the app on the emulator, enter matching or non-matching text, and verify the output.

Result:

The Android application using Jetpack compose was successfully created & executed on the emulator. State management using remember and mutableStateOf worked correctly. The string comparison logic functioned as expected and the UI updated reactively based on the input. The composable structure & theming were applied without errors.

App Navigation - Activity Transition between Multiple Screens in Android

Aim:-

To demonstrate multi-screen navigation in an Android application by implementing bi-directional transitions between two activities using Android Intents, helping to understand the concept of Activity stack & navigation flow.

Algorithm

1. open Android Studio and create a new Android project with the Empty Activity template.
2. Name the project and set the package name as com.example.lab3.
3. Open activity-main.xml and design the main screen layout using Jetpack Compose or XML
4. Add a title text composable or TextView displaying LAB 3 and a subtitle App Navigation.
5. Add a Button labeled Go to Second Page and a descriptive Text below it saying Click the button above to navigate to another page
6. Create a new kotlin file named SecondActivity.kt extending Component Activity or AppCompatActivity.
7. Design the second screen layout with a title second page and subtitle Welcome to the second page
8. Add a Button Go Back to Main and descriptive Text below it
9. In main Activity, set an onClick Listener on the first button.

10. Inside the click handler, create an ~~Intent~~ Intent object with the current context & SecondActivity class as the target.

11. call startActivity() & set an OnClickListener on the Go Back to Main Button

12. In Second Activity, set an OnClickListener on the Go Back to Main button

13. call finish() inside the click handler to pop Second Activity off the back stack & return to main Activity.

14. Open AndroidManifest.xml and verify that both MainActivity & SecondActivity are registered under the application tag

by
Result: The Android application was successfully created with two activities. Navigation from MainActivity to SecondActivity using finish() also worked as expected. Both activities were correctly registered in AndroidManifest.xml. The Activity back stack was maintained properly throughout the navigation flow.

Activity & Fragment Lifecycles - simple Timer with Lifecycle event logging

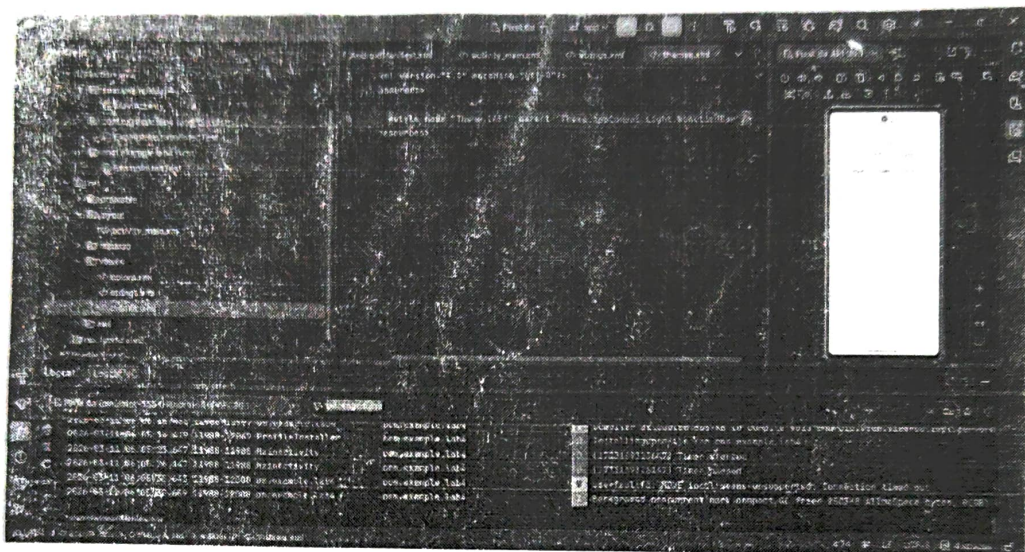
Aim

To understand the Android Activity & Fragment lifecycle by building a simple Timer application that logs each lifecycle callback event with timestamps, demonstrating how the system manages Activity states during user interactions.

Algorithm:

1. Open Android Studio & create a new Android project using the Empty Activity template with XML layouts.
2. Name the project & set the package name as com.example.lch.
3. Open Activity-main.xml & design the UI layout.
4. Add a Text view at the top to display the app title LAB 4 & subtitle Activity & Fragment lifecycle.
5. Add another text view labeled simple timer to serve as the timer section.
6. Add a text view to display the timer value in format.
7. Add three buttons in a horizontal row labelled START, PAUSE, STOP.
8. Add a Text view labelled Activity Lifecycle Events as a section header below the buttons.
9. Add a ScrollView containing a text view to display the lifecycle log entries.
10. In MainActivity.kt, declare a Handler and a Runnable to update the timer every millisecond.
11. Declare variable to track elapsed time, running state & start time.
12. In the start button click listener, record the start time & post the runnable to the handler.

2010-01-20 10:00:00 (17110A)
2010-01-20 10:00:00 (17110A)



13. In the PAUSE button click listener, remove the runnable from the Handler & save elapsed time
14. In the STOP button click listener, stop the timer & reset the elapsed time to zero.
15. override all lifecycle methods including onCreate(), onStart(), onResume(), onPause(), onStop() & onDestroy()
16. Inside each overridden method, call Log.d() with a tag and a message containing timestamp
17. Build & run the app on the emulator interact with the timer buttons & observe the lifecycle logs.

Result: The Android app successfully executed a timer & verified all 6 lifecycle methods. Events were logged to Logcat & the UI, confirming correct behaviour during launch & backgrounding

Jetpack Compose UI using Scaffold Layout

Aim

To develop an Android application using Jetpack compose & Scaffold layout to organize UI using compose's functions

Algorithm:

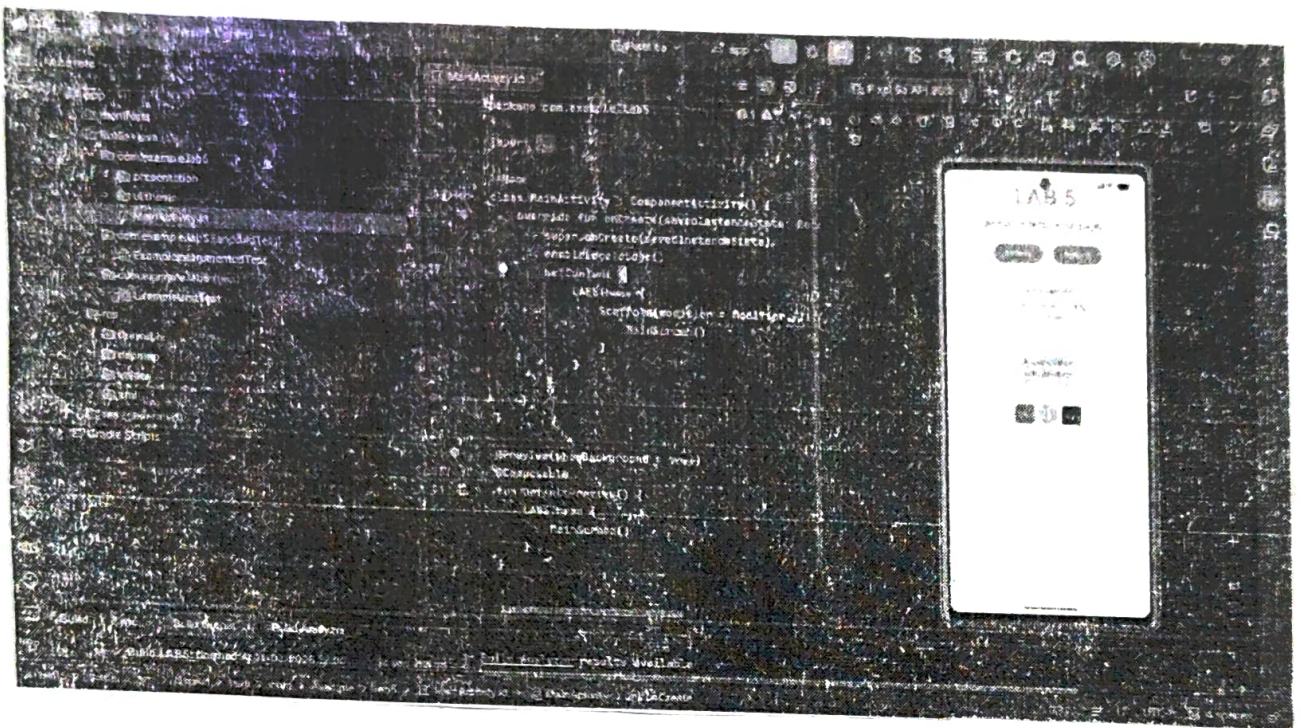
1. open Android Studio & create a new project using empty compose Activity.
2. Set the package name as `*com.example.lab5*`.
3. open `MainActivity.kt` and extend `ComponentActivity`
4. override the `onCreate()` method
5. Enable edge to edge display using `enableEdgeToEdge()`.
6. use `setContent()` to define the compose UI
7. Apply `LAB5Theme` for styling
8. use scaffold as the main layout container
9. Call `mainScreen()` composable inside scaffold
10. Build and run the application

Output

The application displays a composable UI screen structured using scaffold and `MainScreen` composable.

Result

The Android application was successfully created using Jetpack Compose UI and scaffold layout



compose UI Implementation with scaffold Layout

Aim

To design an Android UI using Jetpack compose with scaffold structure & composable screen

Algorithm

1. Create a new Empty Compose Activity Project
2. Set package name com.example.lab6
3. Open MainActivity.kt and extend ComponentActivity.
4. Override onCreate()
5. Enable edge to edge display
6. Use setContent {} to define Compose UI
7. Apply LAB6 Theme
8. Add scaffold layout with fillMaxSize()
9. Click | call mainScreen() composable
10. Run the application

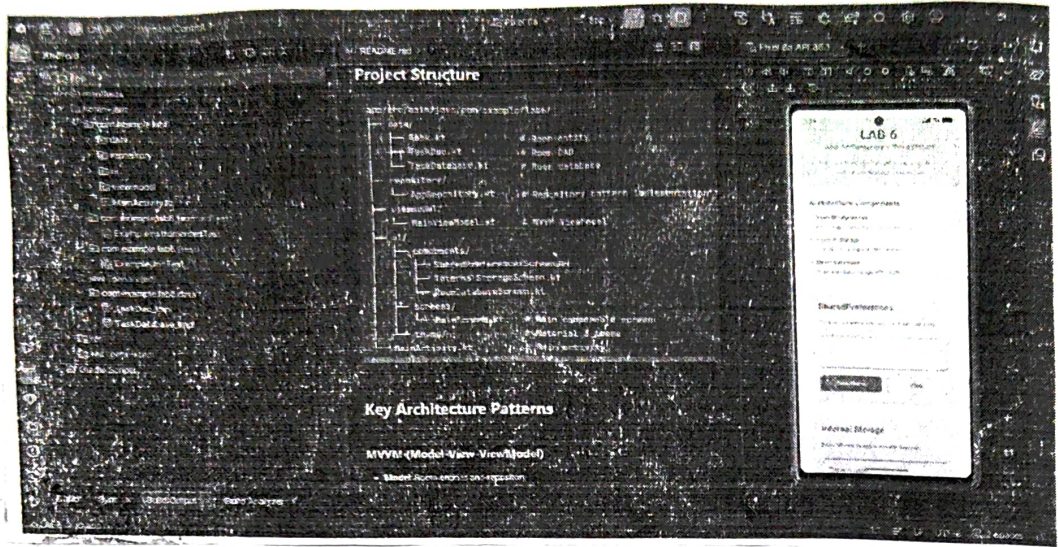
Output

The screen displays UI content using Jetpack compose & scaffold layout.

by

Result:

The application executed successfully & displayed the compose UI using a structured layout



Recyclerview Implementation with Custom Adapter

Aim :-

To develop an Android application using Recyclerview with custom adapter to display a list of items.

Algorithm

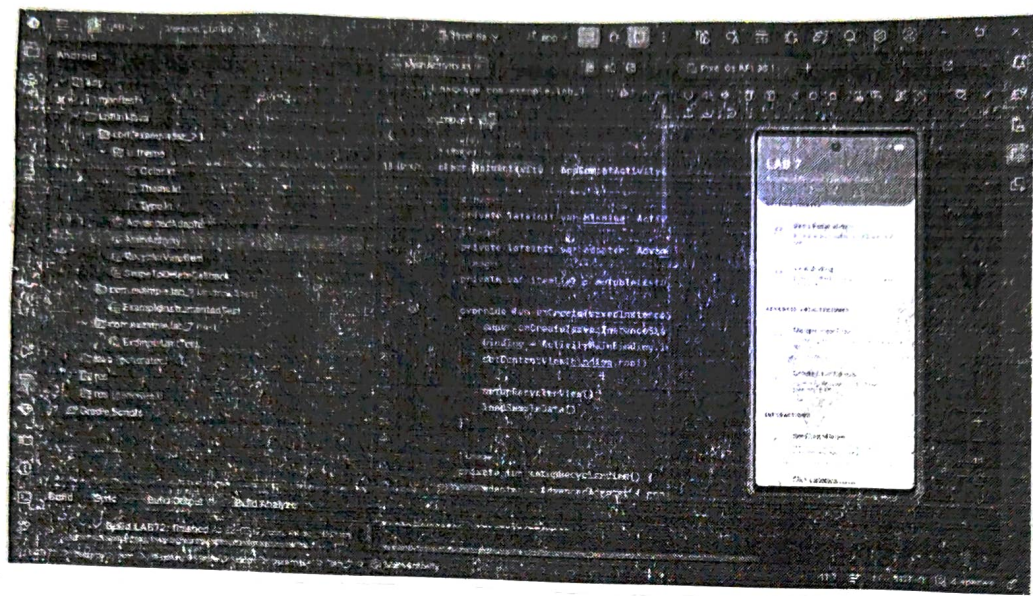
1. Create a new Android project
2. Add a RecyclerView in the layout file.
3. Create a data model class for list items.
4. Create a RecyclerView Adapter with ViewHolder.
5. Override adapter methods (onCreateViewHolder, onBindViewHolder, getItemCount).
6. Initialize RecyclerView in MainActivity
7. Set LinearLayoutManager for vertical list layout
8. Assign the adapter to RecyclerView
9. ^{Add} ~~Run~~ the sample data to the list
10. Run the application

Output

The application displays a scrollable list of items using RecyclerView. Clicking an item shows a Toast message

Result

 The RecyclerView successfully displayed data with a custom adapter & handled item click events.



The code above is a simple Android application that displays a list of items. It uses the RecyclerView widget to display the list, and the ArrayAdapter class to manage the data. The code is written in Java and is part of an Android Studio project.

The code is a simple Android application that displays a list of items. It uses the RecyclerView widget to display the list, and the ArrayAdapter class to manage the data. The code is written in Java and is part of an Android Studio project.

The code is a simple Android application that displays a list of items. It uses the RecyclerView widget to display the list, and the ArrayAdapter class to manage the data. The code is written in Java and is part of an Android Studio project.

Internet Connectivity Monitoring using Connectivity Manager

Aim

To create an Android application that detects internet connectivity status in real time using Connectivity Manager

Algorithm

1. Create a new Empty Compose Activity project
2. Set package name com.example.lab8
3. Use Connectivity Manager to access network information
4. Check the current network status
5. Create a Network Callback to detect connectivity changes
6. Register the callback using NetworkRequest
7. Store connectivity status using MutableStateOf
8. Create a ConnectionDisplay for UI
9. Display connection status with icons & text
10. Run the application and test network changes

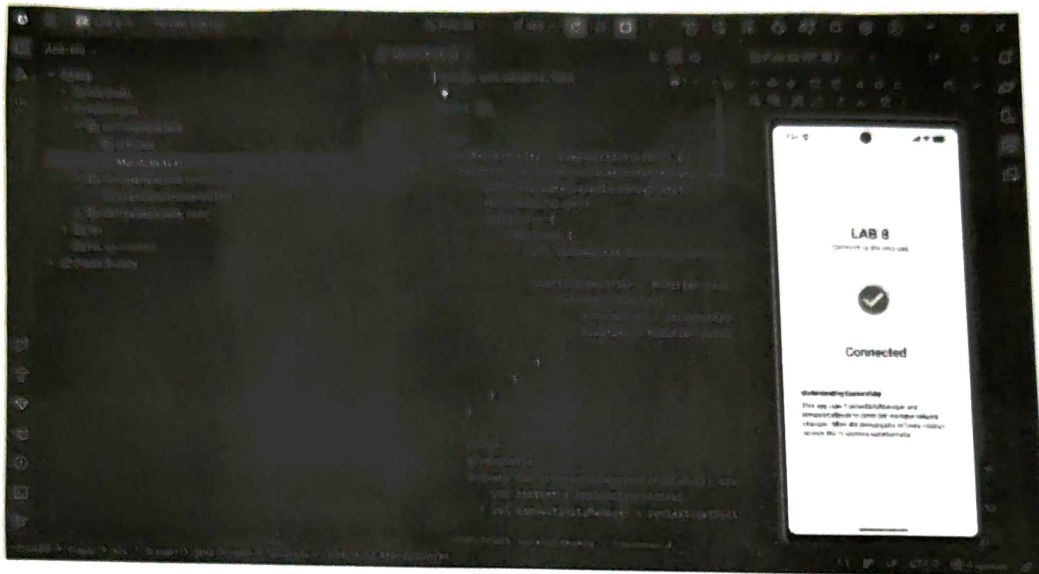
Output:

The screen shows connected or disconnected status with visual indicators depending on internet availability.

By

Result:

The application successfully detected real time internet connectivity changes
→ updated the UI



1. The first step is to connect the device to the network. This can be done by either using a Wi-Fi connection or a cellular data connection. Once the device is connected, the next step is to open the application and navigate to the 'LAB 8' section.

2. The second step is to determine the connectivity status of the device. This can be done by checking the 'Connected' status in the 'LAB 8' section. If the device is connected, the status will be 'Connected'. If the device is not connected, the status will be 'Not Connected'.

3. The third step is to determine the connectivity status of the device. This can be done by checking the 'Connected' status in the 'LAB 8' section. If the device is connected, the status will be 'Connected'. If the device is not connected, the status will be 'Not Connected'.

4. The fourth step is to determine the connectivity status of the device. This can be done by checking the 'Connected' status in the 'LAB 8' section. If the device is connected, the status will be 'Connected'. If the device is not connected, the status will be 'Not Connected'.

5. The fifth step is to determine the connectivity status of the device. This can be done by checking the 'Connected' status in the 'LAB 8' section. If the device is connected, the status will be 'Connected'. If the device is not connected, the status will be 'Not Connected'.

6. The sixth step is to determine the connectivity status of the device. This can be done by checking the 'Connected' status in the 'LAB 8' section. If the device is connected, the status will be 'Connected'. If the device is not connected, the status will be 'Not Connected'.

Repository Pattern with Room Database & Workmanager

Aim

To develop an Android application implementing the Repository pattern with room database and work manager for background data synchronisation

Algorithm

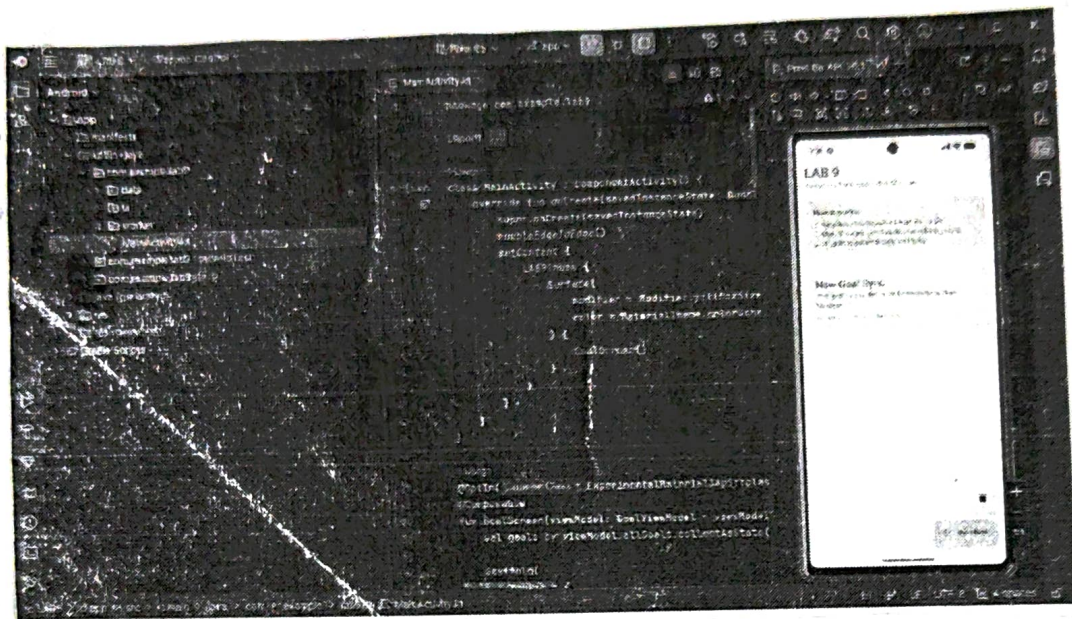
1. create a new compose Activity project
2. Set package name com.example.lab9
3. Create a Goal data model class
4. Implement the Room Database & DAO for data storage.
5. Create a Repository to manage data operations
6. Implement Goalview Model for ui data management
7. Use flow to observe database updates
8. Use workspace Manager to perform background Synchronization
9. Display goals using LazyColumn in compose
10. Run the application

Output

The application displays a list of goals with title, description & timestamp users can sync or delete goals

Result

The application successfully demonstrated repository architecture with Room Database & WorkManager.



Navigation between Multiple screens using navigation compose

Aim

To implement multi-screen navigation in Jetpack compose using the Navigation component.

Algorithm

1. create a new compose Activity project
2. Add Navigation compose dependency
3. create different screen composables
4. Define navigation routes using a Screen class
5. Initialize NavController using `rememberNavController()`
6. create NavHost with a start destination
7. Add composable routes for each screen.
8. Navigate between screens using `navcontroller.navigate()`
- a. Implement back navigation using `popBackStack()`
10. Run the application

Output

The application displays a home screen with options to navigate to multiple screens demonstrating compose navigation

Result

The application successfully implemented multi-screen navigation using Jetpack Compose navigation



